WEEK 4 · LESSON 22 OF 24 · HANDOUT

Open Models and Running AI Locally

Study time: **about 55 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=sYliwvml9Es — open the lesson online to watch inside the course.

🎓 **Go deeper:** EuroPython 2024: Fine-tuning large models on local hardware — https://www.youtube.com/watch?v=hdHKCjhF68U

🎓 **Go deeper:** Andrej Karpathy: Intro to Large Language Models (1-hour talk) — https://www.youtube.com/watch?v=zjkBMFhNj_g

Your goal in this lesson: Explain open-weight models, estimate the memory a model needs, understand quantization, run an open model with Hugging Face transformers and Ollama, and choose between an API and a self-hosted model.

Open-weight models

An **open-weight** model's trained weights can be downloaded and run on your own computer or server. Popular families include Llama, Mistral, Gemma, Qwen, DeepSeek and the small SmolLM models. Sizes range from about 100 million to hundreds of billions of parameters.

Read the **licence**: some allow any use, some restrict commercial use or very large companies, and some require attribution.

Why run a model yourself?

	API model	Self-hosted open model
Quality	Usually the best available	Good, improving fast; small models are weaker
Cost	Pay per token	Pay for hardware and electricity, any number of tokens
Privacy	Data goes to the provider (check their terms)	Data can stay on your own machine
Offline	No	Yes
Work for you	Almost none	Setup, updates, scaling, monitoring

For a school with poor internet, sensitive records, or millions of simple requests, a small local model can be the right choice. For the hardest tasks, top API models are usually much better.

How much memory does a model need?

Each parameter is stored as a number. Its size depends on the precision:

```
BYTES = {"float32": 4, "float16": 2, "int8": 1, "4-bit": 0.5}

def memory_gb(params_billion, precision):
    return params_billion * 1e9 * BYTES[precision] / 1e9

print(f"{'model':>6s}" + "".join(f"{p:>10s}" for p in BYTES))
for size in [0.135, 1, 8, 70]:
    print(f"{size:>5}B" + "".join(f"{memory_gb(size, p):>9.1f}G" for p in BYTES))
```

An 8-billion-parameter model needs about **16 GB** in float16 but only about **4 GB** in 4-bit — the difference between needing a big GPU and fitting on a laptop. (Running also needs extra memory for the context, so add some room.)

Quantization: smaller numbers, almost the same model

Quantization stores weights with fewer bits. Here is 8-bit quantization of one weight matrix:

```
import numpy as np

rng = np.random.default_rng(0)
W = rng.normal(0, 0.02, size=(512, 512)).astype("float32")

scale = np.abs(W).max() / 127                    # map the largest weight to 127
W_int8 = np.round(W / scale).astype("int8")      # store small integers
W_back = W_int8.astype("float32") * scale        # convert back when computing

x = rng.normal(size=512).astype("float32")
rel_error = np.linalg.norm(W @ x - W_back @ x) / np.linalg.norm(W @ x)
print(f"size: {W.nbytes / 1024:.0f} KB -> {W_int8.nbytes / 1024:.0f} KB, output error:
{rel_error:.2%}")
```

Four times smaller, with an output error around **1%**. Real 4-bit methods (such as GPTQ, AWQ and the GGUF formats used by llama.cpp and Ollama) use cleverer scales per small group of weights to keep quality high.

Run an open model with Hugging Face transformers

In Colab (a free GPU helps but this tiny model also runs on CPU):

```
from transformers import AutoTokenizer, pipeline

MODEL_ID = "HuggingFaceTB/SmolLM2-135M-Instruct"        # 135 million parameters, about 270 MB
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
generator = pipeline("text-generation", model=MODEL_ID, tokenizer=tokenizer)

messages = [{"role": "user", "content": "Give one tip for saving money as a student."}]
print(tokenizer.apply_chat_template(messages, tokenize=False, add_generation_prompt=True))

out = generator(messages, max_new_tokens=60, do_sample=False)
print(out[0]["generated_text"][-1]["content"])
```

The printed template shows the special tokens that mark each turn — the **chat template** you built yourself in Lesson 5. A 135-million-parameter model gives simple, sometimes wrong answers; try a 1–3 billion parameter model on a Colab GPU for much better results.

Run models with Ollama

Ollama downloads quantized models and runs them on your laptop or server with one command:

```
# on your own computer (see ollama.com for the installer)
ollama pull llama3.2:1b
ollama run llama3.2:1b "Explain mobile money in one sentence."
```

Ollama also offers an **OpenAI-compatible API** on your machine, so the code from Week 2 works with only a different address:

```
from openai import OpenAI

local = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")   # the key is not
checked
reply = local.chat.completions.create(model="llama3.2:1b",
                                      messages=[{"role": "user", "content": "Say hello in
Kinyarwanda."}])
print(reply.choices[0].message.content)
```

Because the interface is the same, you can **start with an API model and later switch** some requests to a local model — or the other way round — without rewriting your app. Keep the evaluation set from Lesson 17 to check that quality is still good enough after switching.

Small models: what to expect

- Good at: classification, extraction, short rewriting, simple chat, and tasks you have fine-tuned them for (Lesson 20).
- Weak at: long reasoning, rare facts, less common languages such as Kinyarwanda, and following many rules at once.
- Always measure on **your** test set before choosing.

Key terms

- **Open-weight model** — a model whose weights you can download and run.
- **Precision** — how many bits store each number (32, 16, 8, 4).
- **Quantization** — storing weights with fewer bits to save memory.
- **Hugging Face transformers** — the main Python library for open models.
- **Ollama** — a tool that runs quantized models locally with an OpenAI-compatible API.
- **Self-hosting** — running the model on hardware you control.

Remember this: Open models can run on your own hardware for privacy, offline use and fixed cost; quantization makes them fit, and OpenAI-compatible interfaces let you switch between local and API models while your evaluation set guards quality.

Name: Date:

Activity

Task 1

Use `memory_gb` to find the largest model that fits in 16 GB in 4-bit.

Task 2

Repeat the quantization demo with 4 bits (integers from −7 to 7). How big is the error now?

Task 3

In Colab, run SmolLM2 with three different questions. Where does it make mistakes?

Task 4

If you have a laptop with 8 GB or more of memory, install Ollama and compare `llama3.2:1b` with an API model on five questions from your evaluation set.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What is an open-weight model?

   A. A model whose trained weights you can download and run yourself
   B. A model without a licence
   C. A model with no weights
   D. Any free chat website

2. About how much memory do the weights of an 8B model need in 4-bit?

   A. About 32 GB
   B. About 400 GB
   C. About 1 MB
   D. About 4 GB

3. What does quantization do?

   A. Deletes layers
   B. Translates the model
   C. Stores weights with fewer bits to save memory, with a small loss of accuracy
   D. Adds parameters

4. Which is a good reason to self-host a model?

   A. Sensitive data must stay on your own machines
   B. You want no setup work
   C. It is always cheaper for ten requests
   D. You need the absolute best quality for hard reasoning

5. Why is Ollama's OpenAI-compatible API useful?

   A. It encrypts data
   B. It needs no models
   C. It is faster than everything
   D. The same code can switch between local and API models

6. What should you do before switching your app to a small local model?

   A. Nothing
   B. Delete your prompts
   C. Remove the tests
   D. Measure it on your evaluation set

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.